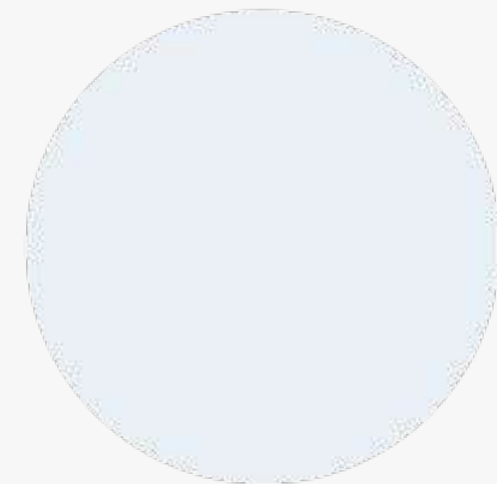


ASTON.



Парадигма ООП



astondevs.ru

Классы



Класс определяет форму и сущность объекта и является логической конструкцией, на основе которой построен весь язык Java.

Наиболее важная особенность класса состоит в том, что он определяет новый тип данных, которым можно воспользоваться для создания объектов этого типа, т.е. класс — это шаблон (чертеж), по которому создаются объекты (экземпляры класса).

Для определения формы и сущности класса указываются данные, которые он должен содержать, а также код, воздействующий на эти данные.

Если мы хотим работать в нашем приложении с документами, то необходимо для начала объяснить что такое документ, описать его в виде класса (чертежа) Document. Рассказать какие у него должны быть свойства: название, содержание, количество страниц, информация о том, кем он подписан и т.д.

В этом же классе мы описываем что можно делать с документами: печатать в консоль, подписывать, изменять содержание, название и т.д. Результатом такого описания и будет наш класс Document.

Классы



Однако это по-прежнему всего лишь чертеж. Если нам нужны конкретные документы, то необходимо создавать объекты: документ №1, документ №2, документ №3. Все эти документы будут иметь одну и ту же структуру (название, содержание, ...), с ними можно выполнять одни и те же действия, НО наполнение будет разным (в первом документе содержится приказ о назначении работника на должность, во втором, о выдаче премии отделу разработки и т.д.).

NB: Заметка. Приведенный пример является абстрактным, и нет необходимости говорить о том, что структура таких документов будет совершенно разной.

Посмотрим на упрощенную форму объявления класса:

```
модификатор_доступа class имя_класса {  
    модификатор_доступа тип_переменной  
    имя_поля; // первое поле  
    модификатор_доступа тип_переменной  
    имя_поля; // второе поле  
    // ...  
    модификатор_доступа тип_переменной  
    имя_поля; // n-е поле  
    модификатор_доступа имя_конструктора  
    (список_аргументов) {  
        // ...  
    }  
    модификатор_доступа тип_метода имя_метода  
    (список_аргументов) {  
        // тело метода  
    }  
    // ...  
    модификатор_доступа тип_метода имя_метода  
    (список_аргументов) {  
        // тело метода  
    }  
}
```

Классы



Как правило, переменные, объявленные в классе, описывают свойства будущих объектов, а методы - их поведение. Например, в классе User (пользователь) можно объявить переменные: int id, String name, String position, int age; которые говорят о том, что у пользователя есть идентификационный номер (id), имя (name), должность (position) и возраст (age). Методы info() и changePosition(), объявленные в классе User, означают что мы можем выводить информацию о нем в консоль (info) и изменять его должность (changePosition).

Переменные, объявленные в классе, называются полями экземпляра, каждый объект класса содержит собственные копии этих переменных, и изменение значения поля у одного объекта никак не повлияет на это же поле другого объекта.

Поля экземпляра и методы, определённые в классе, называются членами класса. В большинстве классов действия над полями осуществляются через его методы.

Пример класса User:

```
public class User {  
    private int id;  
    private String name;  
    private String position;  
    private int age;  
    public User(int id, String name, String  
position, int age) {  
        this.id = id;  
        this.name = name;  
        this.position = position;  
        this.age = age;  
    }  
    public void info() {  
        System.out.println("id: " + id + "; Имя  
пользователя: " +  
        name + "; должность: " + position + ";  
возраст: " + age);  
    }  
    public void changePosition(String position) {  
        this.position = position;  
        System.out.println("Пользователь " + name  
+  
        " получил новую должность: " + position);  
    }  
}
```

Классы



Представим, что нам необходимо работать в нашем приложении с кошками. Java ничего не знает о том, что такое коты, поэтому нам необходимо создать новый класс (тип данных), и объяснить что же такое кот. Для этого начнем потихоньку прописывать класс Cat. Пусть у котов есть три свойства: name (кличка), color (цвет) и age (возраст); и они пока ничего не умеют делать.

```
public class Cat {  
    String name;  
    String color;  
    int age;  
}
```

NB: Важно! Имя класса должно совпадать с именем файла, в котором он объявлен, т.е. класс Cat должен находиться в файле Cat.java

Итак, мы рассказали Java что такое коты, теперь если мы хотим создать в нашем приложении кота, следует воспользоваться следующим оператором.

```
Cat cat1 = new Cat();
```

Мы создали объект типа Cat (экземпляр класса Cat), и для того, чтобы с ним работать, положили его в переменную, которой присвоили имя cat1. На самом деле, в переменной не лежит весь объект, а только ссылка где его искать в памяти, но об этом позже.

Объект cat1 создан по “чертежу” Cat, и значит у него есть поля name, color, age, с которыми можно работать (получать или изменять их значения).

Для доступа к полям объекта служит операция-точка, которая связывает имя объекта с именем поля.

Например, чтобы присвоить полю color объекта cat1 значение “White”, нужно выполнить следующий оператор:

```
cat1.color = "Белый";
```

Операция-точка служит для доступа к полям и методам объекта по его имени.

Классы



Рассмотрим пример консольного приложения, работающего с объектами класса Cat.

```
public class CatDemoApp {  
    public static void main(String[] args) {  
        Cat cat1 = new Cat();  
        Cat cat2 = new Cat();  
        cat1.name = "Барсик";  
        cat1.color = "Белый";  
        cat1.age = 4;  
        cat2.name = "Мурзик";  
        cat2.color = "Черный";  
        cat2.age = 6;  
  
        System.out.println("Кот1 имя: " + cat1.name + " цвет:  
" +  
        cat1.color + " возраст: " + cat1.age);  
        System.out.println("Кот2 имя: " + cat2.name + " цвет:  
" +  
        cat2.color + " возраст: " + cat2.age);  
    }  
}
```

Результат работы программы:

```
Кот1 имя: Барсик цвет: Белый возраст: 4  
Кот2 имя: Мурзик цвет: Черный возраст: 6
```

Вначале мы создали два объекта типа Cat: cat1 и cat2, соответственно они имеют одинаковый набор полей (name, color, age), однако каждому из них мы в эти поля записали разные значения.

Как видно из результата печати в консоли, изменение значения полей одного объекта, никак не влияет на значения полей другого объекта. Данные объектов cat1 и cat2 изолированы друг от друга.

Внутренние (inner) классы



Нестатические вложенные классы принято называть внутренними, или inner классами.

Связь между внешним и внутренним классами при этом определяется необходимостью привязывания логической сущности внутреннего класса к сущности внешнего класса.

Доступ к элементам внутреннего класса возможен из внешнего только через объект внутреннего класса, который должен быть создан в коде метода внешнего класса. Объект внутреннего класса всегда ассоциируется (скрыто хранит ссылку) с создавшим его объектом внешнего класса — так называемым внешним, или enclosing объектом.

Пусть изначально описание класса Student представлено в виде:

```
public class Student {  
    private int studentId;  
    private String name;  
    private int group;  
    private String faculty;  
    private String city;  
    private String street;  
    private int houseId;  
    private int flatId;  
    private String email;  
    private String skype;  
    private long phoneNumber;  
    // constructors, methods  
}
```

Внутренние (inner) классы



В описании класса можно целый набор данных объединить под общим именем Address и выделить его как внутренний класс. Класс Student станет более коротким и понятным. Внешний и внутренний классы могут выглядеть в итоге так:

```
public class Student {  
    private int studentId;  
    private String name;  
    private int group;  
    private String faculty;  
    private String email;  
    private String skype;  
    private long phoneNumber;  
    // constructors, methods  
    public class Address {  
        private String city;  
        private String street;  
        private int houseId;  
        private int flatId;  
    }  
}
```


Объявление внутреннего класса и использование его в качестве поля



Внутренний класс может быть использован любым членом своего внешнего класса, а может и не использоваться вовсе, хотя в этом случае утрачивается его смысл.

Использование объекта внутреннего класса вне своего внешнего класса возможно только при наличии доступа (видимости) и при объявлении ссылки в виде:

```
Student.Address address = new Student().new  
Address();
```

Здесь сначала создается объект внешнего класса, а затем объект внутреннего класса. Другим способом объект внутреннего класса создать не получится.

Объекту внутреннего класса совершенно не обязательно быть полем класса-владельца. Основное отличие от внешнего класса состоит в больших возможностях ограничения видимости и сокрытия реализации внутреннего класса по сравнению с обычным внешним классом.

Внутренний класс может быть объявлен как **private**, что обеспечивает его полную невидимость вне класса-владельца и надежное сокрытие реализации. В этом случае ссылку `address`, приведенную выше, объявить было бы нельзя.

Создать объект такого класса можно только в методах и логических блоках внешнего класса. Использование **protected** позволяет получить доступ к внутреннему классу для класса в другом пакете, являющегося суперклассом внешнего класса.

Объявление внутреннего класса и использование его в качестве поля



При компиляции внутренний класс получает собственный модуль для интерпретации, соответствующий внутреннему классу, который получит имя `Student$Address.class`. Внешний же класс будет скомпилирован в обычный файл `Student.class`.

Методы внутреннего класса имеют прямой доступ ко всем полям и методам внешнего класса, как будто они его собственные, в то же время внешний класс может получить доступ к содержимому внутреннего класса только после создания объекта внутреннего класса.

Доступ будет разрешен по имени, в том числе и к полям, объявленным как **private**. Внутренние классы не могут содержать статические поля и методы, кроме `final static`.

Доступ к полям внешнего класса



```
public class Owner {  
    private int id;  
    public class Inner {  
        public void buildId() {  
            id += 1000;  
        }  
    }  
}
```

Внешний класс напрямую не видит никаких компонентов своего внутреннего класса.
Вопрос доступа к полям внешнего класса можно рассмотреть путем объявления полей с одинаковыми именами во внутреннем и внешнем классах, чего на практике делать не рекомендуется.

Некорректное объявление полей классов



```
public class DumberOwner {  
  
    private int id;  
    public class DumberInner {  
        private int id;  
  
        public void buildId(int id) {  
            this.id = id + 100 *  
DumberOwner.this.id;  
        }  
    }  
}
```

Поле `id` объявлено как поле класса `DumberInner` и при попытке доступа к полю `id` класса `DumberOwner` обращения типа `id` или `this.id` приведут к обращению к полю внутреннего класса. Для доступа к `id` внешнего класса нужно указать на имя класса, которому принадлежит объект, а именно `DumberOwner.this.id`.

Внутренний класс может быть объявлен также внутри метода или логического блока внешнего (owner) класса. Видимость такого класса регулируется областью видимости блока, в котором он объявлен.

Но внутренний класс сохраняет доступ ко всем полям и методам внешнего класса, а также ко всем константам, объявленным в текущем блоке кода.

Класс, объявленный внутри метода, не может быть объявлен как `static`, а также не может содержать статические поля и методы.

Внутренний класс, объявленный внутри метода



```
public abstract class AbstractTeacher {
    private int id;
    public AbstractTeacher(int id) {
        this.id = id;
    }
    public abstract boolean remandStudent(Student student);
}

public class Teacher extends AbstractTeacher {
    public Teacher(int id) {
        super(id);
    }
    @Override
    public boolean remandStudent(Student student) {
        return false;
    }
}

public class TeacherCreator {
    public static AbstractTeacher createTeacher(int id) {
        int value = 0;
        // class declaration inside a method
        class Rector extends AbstractTeacher {
            Rector(int id) {
                super(id);
            }
        }
        @Override
        public boolean remandStudent(Student student) {
            // value++; compile error
            boolean result = false;
        }
    }
}
```

```
if (student != null) {
    // student status change code in the database
    result = true;
}
return result;
} // inner class: end
if (isRectorId(id)) {
    return new Rector(id);
} else {
    return new Teacher(id);
}

private static boolean isRectorId(int id) { // checking id in the
database
    return (id == 6); // stub
}

public class TeacherLogic {
    public void expelledProcess(int rectorId, Student student) {
        AbstractTeacher teacher =
TeacherCreator.createTeacher(rectorId);
        boolean result = teacher.remandStudent(student);
        System.out.println("Student expelled: " + result);
    }
}
```


Внутренний класс, объявленный внутри метода



```
public class StudyMain {  
    public static void main(String[] args) {  
        TeacherLogic logic = new TeacherLogic();  
        Student student = new Student();  
        logic.expelledProcess(42, student);  
        logic.expelledProcess(6, student);  
    }  
}
```

В результате будет выведено:

Student expelled: false

Student expelled: true

Однако существует единственная возможность получить ссылку на класс, объявленный внутри метода, и использовать его специфические свойства, как в данном случае, при наследовании внутренним классом функциональности обычного класса, в частности, `AbstractTeacher`. При компиляции данного кода с внутренним классом ассоциируется объектный модуль со сложным именем `TeacherCreator$1Rector.class`, тем не менее однозначно определяющим связь между внешним и внутренним классами. Цифра 1 в имени говорит о том, что в других методах класса также можно объявить внутренний класс с таким же именем.

Класс `Rector` объявлен в методе `createTeacher(int id)` и, соответственно, объекты этого класса можно создавать только внутри метода, из любого другого метода или конструктора внешнего класса внутренний класс недоступен.

Свойства внутренних классов:



- Доступ к элементам внутреннего класса возможен только из внешнего класса через объект внутреннего класса;
- Методы внутреннего класса имеют прямой доступ ко всем полям и методам внешнего класса;
- Объект внутреннего класса имеет ссылку на объект своего внешнего класса (enclosing);
- Внутренние классы не могут содержать static-полей и методов, кроме final static;
- Внутренние классы могут быть производными от других классов;
- Внутренние классы могут быть суперклассами;
- Внутренние классы могут реализовывать интерфейсы;
- Внутренние классы могут быть объявлены с параметрами final, abstract, private, protected, public;
- Если необходимо создать объект внутреннего класса где-нибудь, кроме внешнего нестатического метода класса, то нужно определить тип объекта как OwnerType.InnerType;
- Внутренний класс может быть объявлен внутри метода или логического блока внешнего класса, видимость класса регулируется видимостью того блока, в котором он объявлен. Однако внутренний класс сохраняет доступ ко всем полям и методам внешнего класса, а также final-переменным, объявленным в текущем блоке кода;
- Локальному внутреннему классу, объявленному внутри метода или логического блока, модификатор доступа не требуется, так как он все равно не доступен напрямую вне метода.

Вложенные (nested) классы



Если не существует жесткой необходимости в одновременном обязательном существовании объекта внутреннего класса и объекта внешнего класса, то есть смысл сделать такой внутренний класс статическим, который будет тогда называться вложенным, или nested классом.

Связь между внешним и внутренним классами определяется необходимостью привязывания логической функциональности внутреннего класса.

Вложенный класс логически связан с классом-владельцем, но его объект может быть использован независимо от объекта внешнего класса. Такой класс обычно определяет дополнительный функционал для класса-владельца.

При объявлении такого внутреннего класса присутствует служебное слово `static`, и такой класс называется вложенным (nested). Если класс объявлен внутри интерфейса, то он получает спецификаторы `public static` по умолчанию.

Вложенные (nested) классы



Такой класс способен наследовать другие классы, реализовывать интерфейсы и являться объектом наследования для любого класса, обладающего необходимыми правами доступа.

В то же время статический вложенный класс для доступа к не статическим членам и методам внешнего класса должен создавать объект внешнего класса, а напрямую иметь доступ только к статическим полям и методам внешнего класса.

Для создания объекта вложенного класса объект внешнего класса создавать нет необходимости. Подкласс вложенного класса не способен унаследовать возможность доступа к членам внешнего класса, которыми наделен его суперкласс.

Если предполагается использовать внутренний класс в качестве подкласса, следует исключить использование в его теле любых прямых обращений к членам класса-владельца.

```
import java.util.Comparator;
public class Student {
    private int studentId;
    private String name;
    private int group;
    private float averageMark;
    public Student(int studentId, String name, int
group, float averageMark) {
        this.studentId = studentId;
        this.name = name;
        this.group = group;
        this.averageMark = averageMark;
    }
    public String getName() {
        return name;
    }
    public int getGroup() {
        return group;
    }
    public float getAverageMark() {
        return averageMark;
    }
}
```

Вложенные (nested) классы



Такой класс способен наследовать другие классы, реализовывать интерфейсы и являться объектом наследования для любого класса, обладающего необходимыми правами доступа.

В то же время статический вложенный класс для доступа к не статическим членам и методам внешнего класса должен создавать объект внешнего класса, а напрямую иметь доступ только к статическим полям и методам внешнего класса.

Для создания объекта вложенного класса объект внешнего класса создавать нет необходимости. Подкласс вложенного класса не способен унаследовать возможность доступа к членам внешнего класса, которыми наделен его суперкласс.

Если предполагается использовать внутренний класс в качестве подкласса, следует исключить использование в его теле любых прямых обращений к членам класса-владельца.

```
// nested classes
public static class GroupComparator implements
Comparator<Student> {

    @Override
    public int compare(Student o1, Student o2) {
        return o1.group - o2.group;
    }
}

public static class NameComparator implements
Comparator<Student> {

    @Override
    public int compare(Student o1, Student o2) {
        return o1.name.compareTo(o2.name);
    }
}

public static class MarkComparator implements
Comparator<Student> {

    @Override
    public int compare(Student o1, Student o2) {
        return Float.compare(o2.averageMark,
o1.averageMark);
    }
}
}
```


Видимость полей



Статическому классу доступно только статическое содержимое класса-владельца.

Класс, вложенный в интерфейс, по умолчанию статический. На него не накладывается никаких особых ограничений, и он может содержать поля и методы как статические, так и нестатические.

```
public class Owner {  
    private int value = 1;  
    static int statValue = 2;  
    public static class Nested {  
        statValue++;  
        // value++; invisible  
    }  
}
```


Класс, вложенный в интерфейс



Такой внутренний класс использует пространство имен интерфейса. Вызов статического метода выглядит так:

```
Logic.NestedLogic.assign();
```

Свойства вложенных классов:

- Вложенный класс может быть базовым, производным, реализующим интерфейсы;
- Статический вложенный класс для доступа к нестатическим членам и методам внешнего класса должен создавать объект внешнего класса;
- Вложенный класс имеет доступ к статическим полям и методам внешнего класса;
- Подкласс вложенного класса не наследует возможность доступа к членам внешнего класса, которыми наделен его суперкласс;
- Класс, вложенный в интерфейс, статический по умолчанию;
- Статический метод вложенного класса вызывается при указании полного относительного пути к нему.

```
public interface Logic {  
    void doLogic();  
    class NestedLogic { // public static: default  
parameters  
        public long value;  
        public NestedLogic() { /* code */ }  
        public static void assign() { /* code */ }  
        public void accept() { /* code */ }  
    }  
}
```

Объекты



Все в Java является объектом.

Вернее, очень мало чего в Java объектом не является. Например, примитивные типы. Но это скорее редкое исключение, чем правило.

Создание объекта проходит в два этапа. Сначала создается переменная, имеющая интересующий нас тип (в данном случае Cat), в нее мы сможем записать ссылку на будущий объект (поэтому при работе с классами и объектами мы говорим о ссылочных типах данных). Затем необходимо выделить память под наш объект, создать и положить объект в выделенную часть памяти, и сохранить ссылку на этот объект в памяти в нашу переменную.

NB: Заметка. Область памяти, в которой создаются и хранятся объекты, называется кучей (heap).

Для непосредственного создания объекта применяется оператор new, который динамически резервирует память под объект и возвращает ссылку на него, в общих чертах эта ссылка представляет собой адрес объекта в памяти, зарезервированной оператором new.

```
public static void main(String[] args) {  
    Cat cat1;  
    cat1 = new Cat();  
}
```

В первой строке кода переменная cat1 объявляется как ссылка на объект типа Cat и пока ещё не ссылается на конкретный объект (первоначально значение переменной cat1 равно null). В следующей строке выделяется память для объекта типа Cat, и в переменную cat1 сохраняется ссылка на него. После выполнения второй строки кода переменную cat1 можно использовать так, как если бы она была объектом типа Cat. Обычно новый объект создается в одну строку (Cat cat1 = new Cat()).

Объекты



Оператор new динамически выделяет память для нового объекта, общая форма применения этого оператора имеет следующий вид:

```
Имя_класса имя_переменной = new Имя_класса();
```

Имя_класса() в правой части выполняет вызов конструктора данного класса, который позволяет подготовить наш объект к работе.

Рассмотрим еще один пример, в котором создаются новые объекты.

```
public static void main(String[] args) {  
    Cat cat1 = new Cat();  
    Cat cat2 = cat1;  
}
```

На первый взгляд может показаться, что переменной cat2 присваивается ссылка на копию объекта cat1, т.е. переменные cat1 и cat2 будут ссылаться на разные объекты в памяти. Но это не так. На самом деле cat1 и cat2 будут ссылаться на один и тот же объект. Присваивание переменной cat1 значения переменной cat2 не привело к выделению области памяти или копированию объекта, лишь к тому, что переменная cat2 ссылается на тот же объект, что и переменная cat1.

Таким образом, любые изменения, внесённые в объекте по ссылке cat2, окажут влияние на объект, на который ссылается переменная cat1, поскольку это один и тот же объект в памяти.

Mutable vs Immutable объекты



Константы позволяют защитить переменные от изменений, но не в состоянии защитить от изменений объекты, на которые эти переменные ссылаются.

В ответ на эту проблему в Java придумали объекты-константы. Или, как их еще называют, immutable-объекты — неизменяемые объекты.

Кстати, вы даже знаете один такой класс, чьи объекты нельзя менять — String. Объект класса String остается неизменным все время после создания. И как же разработчики Java добились этого?

Mutable vs Immutable объекты



- Во-первых, все переменные класса String скрыты – объявлены private.
- Во-вторых, наследоваться от класса String нельзя: он имеет модификатор final в объявлении класса.
- В-третьих, что самое интересное, все методы класса String, которые по идее должны были менять существующий объект, не меняют его, а возвращают новый.

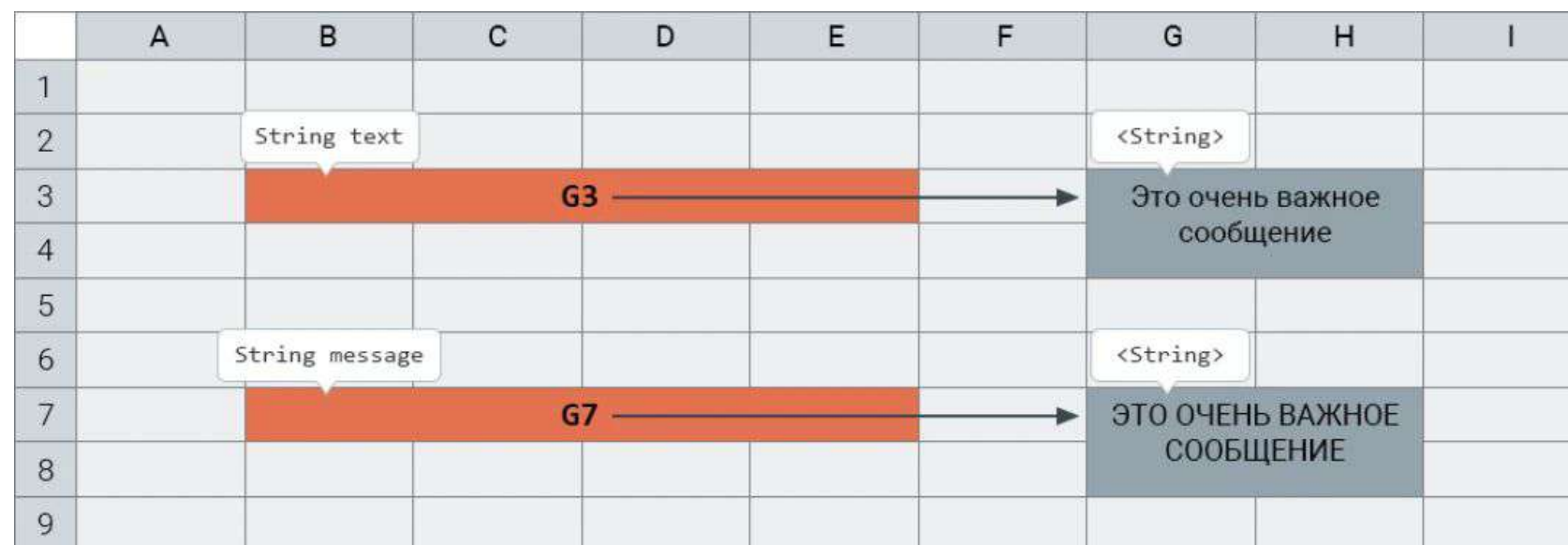
Mutable vs Immutable объекты



Например, метод `toUpperCase()` делает все буквы строки большими (заглавными). Но вместо изменения объекта, у которого он вызван, этот метод возвращает новый объект типа `String`, состоящий из заглавных букв:

```
String text = "Это очень важное сообщение";  
String message = text.toUpperCase();
```

Вот что будет в памяти после выполнения этого кода:



Конструкторы



Давайте еще раз взглянем на один из предыдущих примеров.

```
public class CatDemoApp {  
    public static void main(String[] args) {  
        Cat cat1 = new Cat();  
        cat1.name = "Барсик";  
        cat1.color = "Белый";  
        cat1.age = 4;  
        System.out.println("Кот1 имя: " + cat1.name +  
" цвет: " +  
        cat1.color + " возраст: " + cat1.age);  
    }  
}
```

Чтобы создать объект мы тратим одну строку кода (`Cat cat1 = new Cat()`). Поля этого объекта заполнятся автоматически значениями по-умолчанию (целочисленные - 0, логические - false, ссылочные - null и т.д.).

Нам бы хотелось дать коту какое-то имя, указать его возраст и цвет, поэтому мы прописываем еще три строки кода. В таком подходе есть несколько недостатков: во-первых, мы напрямую обращаемся к полям объекта (чего не стоит делать, в соответствии с принципами инкапсуляции, о которых речь пойдет чуть позже), а во-вторых, если полей у класса будет намного больше, то для создания всего лишь одного объекта будет уходить 10-20+ строк кода, что очень неудобно. Было бы неплохо иметь возможность сразу при создании объекта указывать значения его полей.

Конструкторы



Для инициализации объектов при создании в Java предназначены конструкторы. Имя конструктора обязательно должно совпадать с именем класса, а синтаксис аналогичен синтаксису метода. Если создаться конструктор класса Cat как показано ниже, он автоматически будет вызываться при создании объекта.

Теперь, при создании объектов класса Cat, все коты будут иметь одинаковые имена, цвет и возраст (а именно, белый двухлетний Барсик).

NB: Заметка. Еще раз обращаем внимание, что в строке `Cat cat1 = new Cat();` подчеркнутая часть кода и есть вызов конструктора класса Cat.

```
public class Cat {  
    private String name;  
    private String color;  
    private int age;  
    public Cat() {  
        System.out.println("Это конструктор класса  
Cat");  
        name = "Барсик";  
        color = "Белый";  
        age = 2;  
    }  
}  
  
public class MainClass {  
    public static void main(String[] args) {  
        Cat cat1 = new Cat();  
    }  
}
```

Конструкторы



NB: Важно! У классов всегда есть конструктор. Даже если вы не пропишите свою реализацию конструктора, то Java автоматически создаст пустой конструктор по-умолчанию. Для класса Cat, он будет выглядеть так:

```
public Cat() {}
```

При использовании конструктора из предыдущего примера, все созданные коты будут одинаковыми, пока мы вручную не поменяем значения их полей. Чтобы можно было указывать начальные значения полей наших объектов необходимо создать параметризованный конструктор.

NB: Важно! В приведенном ниже примере, в аргументах конструктора используется нижнее подчеркивание `_`, это сделано для упрощения понимания логики заполнения полей объекта. И в будущем будет заменено на более корректное использование ключевого слова `this`.

```
public class Cat {  
    private String name;  
    private String color;  
    private int age;  
    public Cat(String _name, String _color, int _age)  
    {  
        name = _name;  
        color = _color;  
        age = _age;  
    }  
}
```

Конструкторы



При такой форме конструктора, когда мы будем кота, необходимо будет обязательно указать его имя, цвет и возраст. Набор полей, которые будут заполнены через конструктор, вы определяете сами, то есть вы не обязаны все поля, которые есть в классе записывать в аргументы конструктора.

```
public static void main(String[] args) {  
    Cat cat1 = new Cat("Барсик", "Коричневый", 4);  
    Cat cat2 = new Cat("Мурзик", "Белый", 5);  
}
```

Наборы значения (Барсик, Коричневый, 4) и (Мурзик, Белый, 5) будут переданы в качестве аргументов конструктора (_name, _color, _age), а конструктор уже перезапишет полученные значения в поля объекта (name, color, age).

То есть начальные значения полей каждого из объектов будет определяться тем, что мы передадим ему в конструкторе. Как видите, теперь нам нет необходимости обращаться напрямую к полям объектов, и мы в одну строку можем проинициализировать наш новый объект.

Конструкторы



Иногда требуется, чтобы метод ссылался на вызвавший его объект. Для этой цели в Java определено ключевое слово `this`. Им можно пользоваться в теле любого метода для ссылки на текущий объект, т.е. объект у которого был вызван этот метод.

```
public class Cat {  
    private String name;  
    private String color;  
    private int age;  
    public Cat(String name, String color, int age) {  
        this.name = name;  
        this.color = color;  
        this.age = age;  
    }  
}
```

Эта версия конструктора действует точно так же, как и предыдущая. Ключевое слово `this` применяется в данном случае для того, чтобы отличить аргумент конструктора от поля объекта.

Конструкторы



Наряду с перегрузкой обычных методов возможна перегрузка и конструкторов. Мы можем как не объявлять ни одного конструктора, так и объявить их несколько. Также как и при перегрузке методов, имеет значение набор аргументов, не может быть нескольких конструкторов с одним и тем же набором аргументов.

```
public class Cat {  
    private String name;  
    private String color;  
    private int age;  
    public Cat(String name, String color, int age) {  
        this.name = name;  
        this.color = color;  
        this.age = age;  
    }  
    public Cat(String name) {  
        this.name = name;  
        this.color = "Неизвестно";  
        this.age = 1;  
    }  
}
```

NB: Важно! Как только вы создали в классе свою реализацию конструктора, конструктор по-умолчанию автоматически создаваться не будет. И если вам понадобится такая форма конструктора (в которой нет аргументов, и которая ничего не делает), необходимо будет конструктор по-умолчанию вручную.

Конструкторы



```
public Cat() {}
```

В этом случае допустимы будут следующие варианты создания объектов:

```
public static void main(String[] args) {  
    // Cat cat1 = new Cat(); этот конструктор больше  
    не работает  
    Cat cat2 = new Cat("Барсик");  
    Cat cat3 = new Cat("Мурзик", "Белый", 5);  
}
```

Соответствующий перегружаемый конструктор вызывается в зависимости от аргументов, указываемых при выполнении оператора new.

NB: Важно! Не лишним будет напомнить что у классов всегда есть конструктор, даже если вы не пропишите свою реализацию конструктора.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



1. Создать класс "Сотрудник" с полями: ФИО, должность, email, телефон, зарплата, возраст.
Конструктор класса должен заполнять эти поля при создании объекта.
Внутри класса «Сотрудник» написать метод, который выводит информацию об объекте в консоль.

2. Создать массив из 5 сотрудников.

Пример:

```
// вначале объявляем массив объектов
Person[] persArray = new Person[5];
// потом для каждой ячейки массива задаем объект
persArray[0] = new Person("Ivanov Ivan", "Engineer",
                           "ivivan@mailbox.com", "892312312", 30000,
                           30);
persArray[1] = new Person(...);
...
persArray[4] = new Person(...);
```

ПРАКТИЧЕСКОЕ ЗАДАНИЕ



3. Создать класс Park с внутренним классом, с помощью объектов которого можно хранить информацию об аттракционах, времени их работы и стоимости.